Industrial Internship Report on

” Python Mini Projects”

Prepared by

Mahalakshmi. K

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. My project was **Python Development and Automation**. I developed four different Python applications, including a URL Shortener, File Organizer, Password Manager, and Quiz Game. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solutions for that. It was an overall great experience to have this internship

TABLE OF CONTENTS

1 Preface 3
2 Introduction 4
2.1 About UniConverge Technologies Pvt Ltd 4
2.2 About upskill Campus 9
2.3 Objective 11
2.4 Reference 11
2.5 Glossary 11
3 Problem Statement 12
4 Existing and Proposed solution 13
5 Proposed Design/ Model 14
5.1 High Level Diagram (if applicable) 14
5.2 Low Level Diagram (if applicable) 15
5.3 Interfaces (if applicable) 15
6 Performance Test 16
6.1 Test Plan/ Test Cases ..... **Error! Bookmark not defined.**
6.2 Test Procedure ..... **Error! Bookmark not defined.**
6.3 Performance Outcome ..... **Error! Bookmark not defined.**
7 My learnings 17
8 Future work scope 18

1 Prefac

This 6-week internship focused on mastering Python basics and implementing practical automation tools.

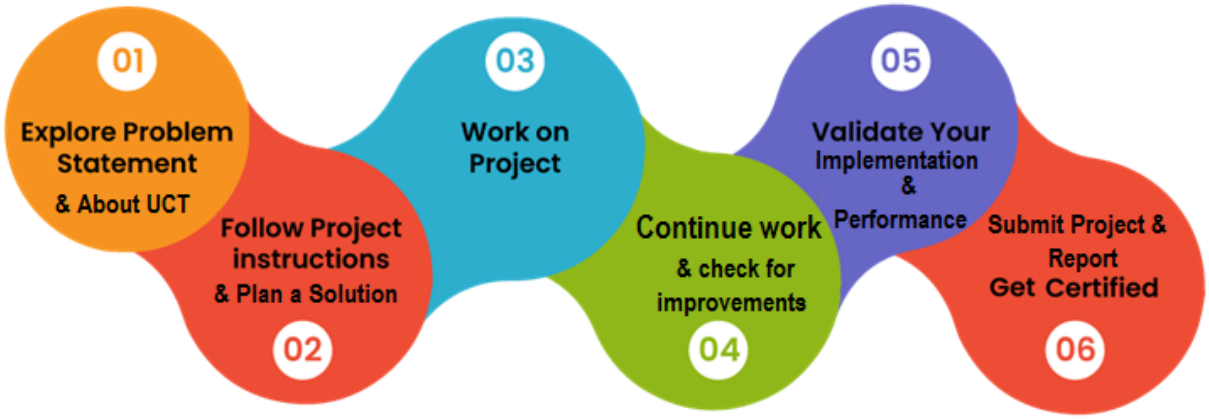
Relevant internship experience is crucial for career development as it bridges the gap between theory and industry.

My project, "Python Mini Projects," addressed common digital tasks like security, file management, and link shortening.

The program was systematically planned with weekly goals.

I learned how to use external libraries and handle real-world debugging.

I want to thank the mentors at USC and UCT for their guidance. To my juniors: focus on hands-on coding and consistent experimentation.



2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI. For developing its products and solutions it is leveraging various Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end etc.

- **i. UCT IoT Platform:** UCT Insight is an IOT platform designed for quick deployment of IOT applications. It has been built in Java for backend and ReactJS for Front end.
- **ii. Smart Factory Platform:** Factory watch is a platform for smart factory needs, providing scalable solutions for production and asset monitoring.
- **iii. LoRaWAN based Solution:** UCT is an early adopter of LoRAWAN technology providing solutions in Agritech, Smart cities, and Industrial Monitoring.
- **iv. Predictive Maintenance:** UCT provides Industrial Machine health monitoring solutions leveraging Embedded systems and Machine Learning.

i. UCT IoT Platform (uct Insight)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process. USC is a career development platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to
• Build Your own dashboard
• Analytics and Reporting
• Alert and Notification
• Integration with third party application(Power BI, SAP, ERP)
• Rule Engine.

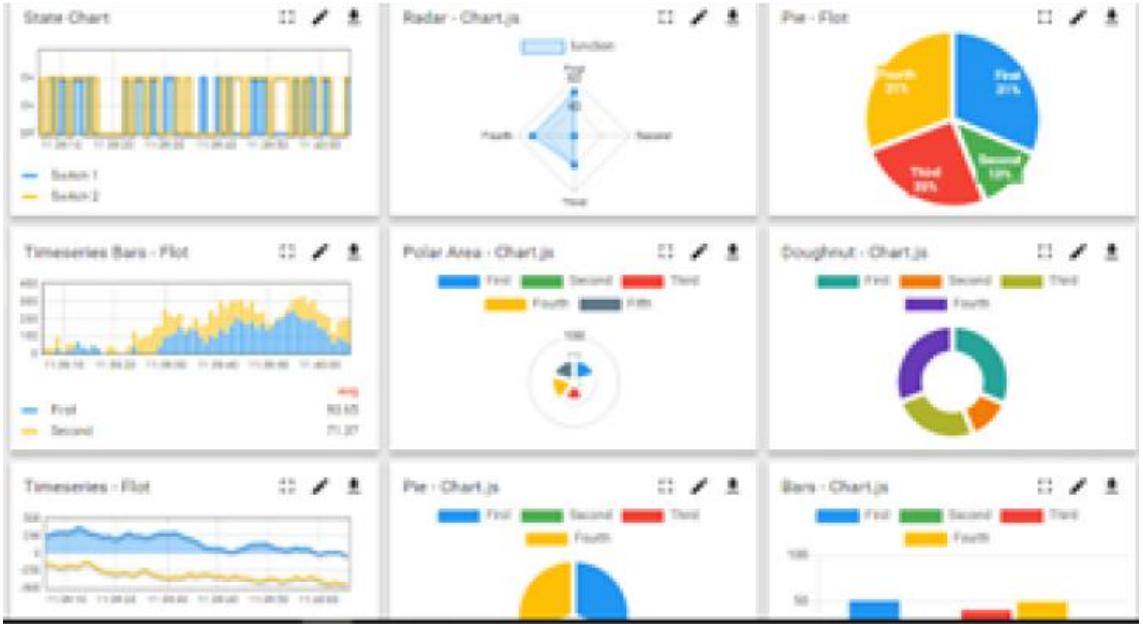
The IoT Academy

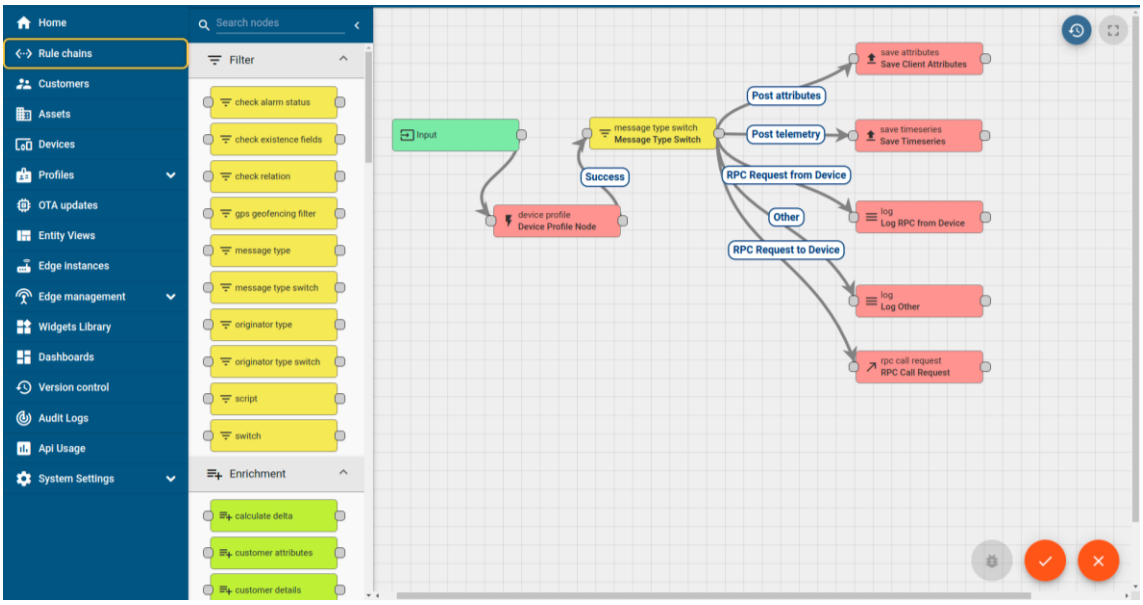
The IoT academy is the EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

Objectives of this Internship program

The objectives for this internship program were:

- To get practical experience of working in the industry.
- To solve real-world problems.
- To have improved job prospects.
- To have an improved understanding of our field and its applications.
- To have personal growth like better communication and problem solving.


State Chart
Radar - Chart.js
Pie - Flot
Timeseries Bars - Flot
Polar Area - Chart.js
Doughnut - Chart.js
Timeseries - Flot
Pie - Chart.js
Bars - Chart.js


Home
Rule chains
Customers
Assets
Devices
Profiles
OTA updates
Entity Views
Edge instances
Edge management
Widgets Library
Dashboards
Version control
Audit Logs
Api Usage
System Settings
Search nodes
Filter
check alarm status
check existence fields
check relation
gps geofencing filter
message type
message type switch
originator type
originator type switch
script
switch
Enrichment
calculate delta
customer attributes
customer details
Input
message type switch
Message Type Switch
Success
device profile
Device Profile Node
Post attributes
Post telemetry
RPC Request from Device
Other
RPC Request to Device
save attributes
Save Client Attributes
save timeseries
Save Timeseries
log
Log RPC from Device
log
Log Other
rpc call request
RPC Call Request

ii. Smart Factory Platform (FACTORY WATCH)

Factory Watch is a platform for smart factory needs. It provides users and factories with:

- A scalable solution for their production and asset monitoring.
- OEE (Overall Equipment Effectiveness) and predictive maintenance solutions scaling up to digital twins for assets.
- The ability to unleash the true potential of the data that machines are generating to help identify and improve KPIs.
- A modular architecture that allows users to choose the service they want to start with and then scale to more complex solutions as per demand. Its unique SaaS model helps users save time, cost, and money.

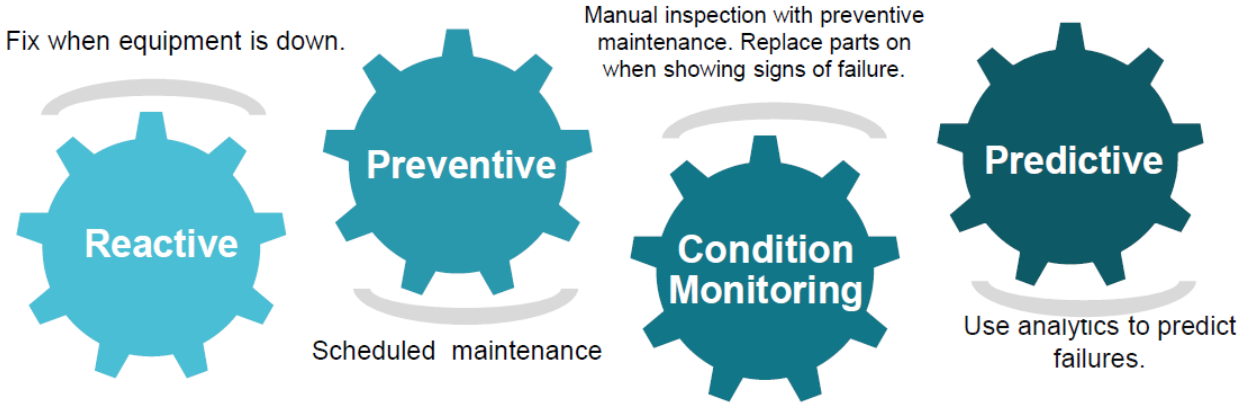
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

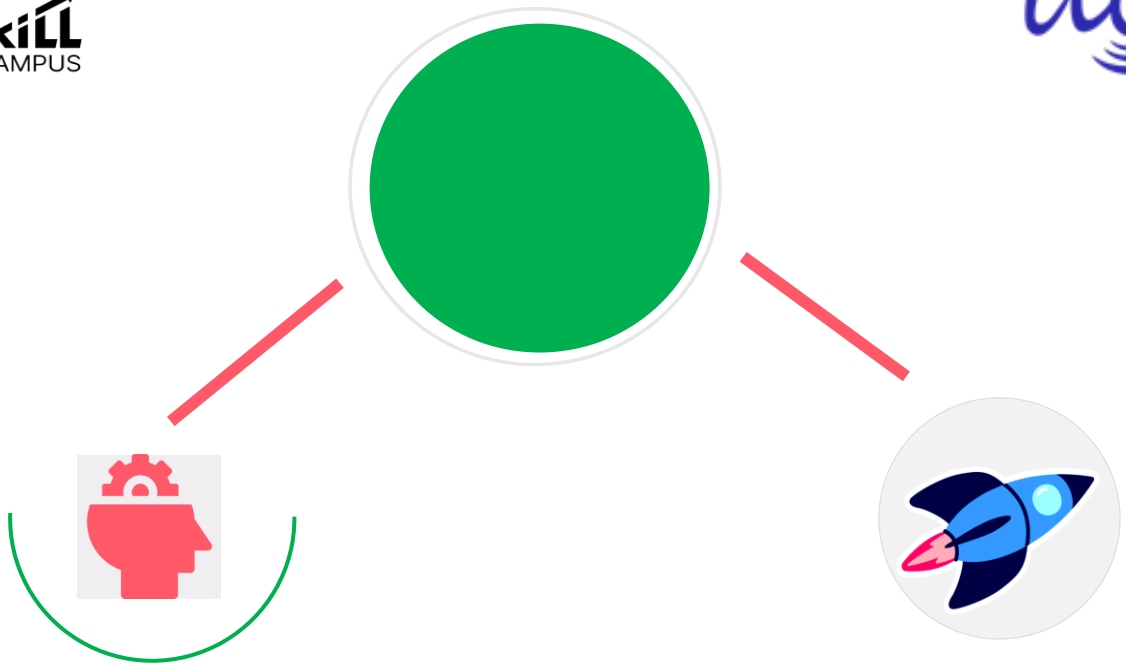
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

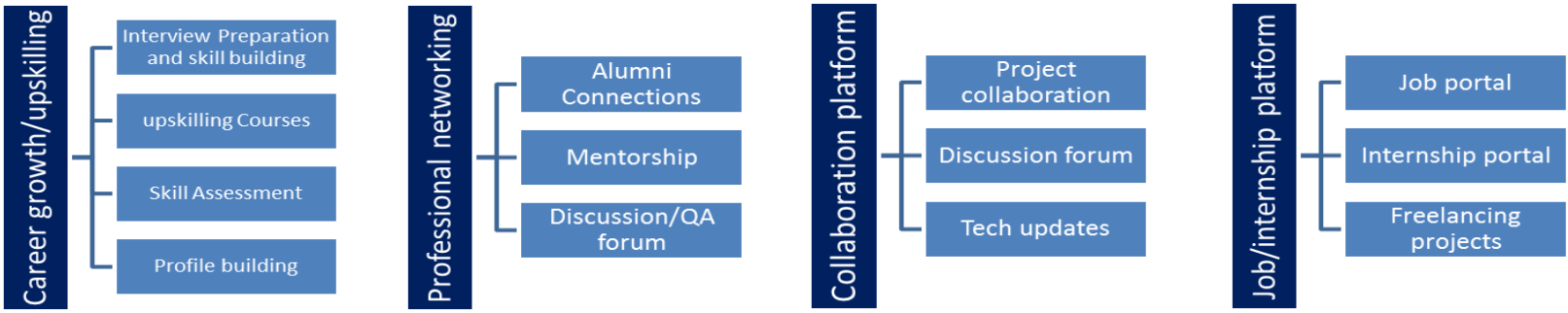
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Python Software Foundation. (2024). *Python 3.12 Documentation*. Retrieved from python.org
[2] Pypi.org. (2024). *Cryptography: Python library for secure data encryption*. Retrieved from pypi.org
[3] Pypi.org. (2024). *Pyshorteners: A Python lib to help you short and expand URLs*. Retrieved from pypi.org
[4] W3Schools. (2024). *Python File Handling and OS Module*. Retrieved from w3schools.com

2.6 Glossary

Term	Acronym / Definition
API	Application Programming Interface (used for the URL Shortener)

IDE	Integrated Development Environment (e.g., VS Code or PyCharm used for coding)
PIP	Package Installer for Python (used to install cryptography and pyshorteners)
GUI	Graphical User Interface (visual buttons and windows)
CLI	Command Line Interface (the terminal where your scripts run)
OS	Operating System (interacted with via Python's `os` module)
UCT	UniConverge Technologies Pvt Ltd
USC	upskill Campus

3 Problem Statement

In the assigned problem statement, the objective was to develop a suite of automation tools using Python to solve common digital inefficiencies. The specific problems addressed were:

- **Link Management:** Long, complex URLs are difficult to share and track.
- **File Disorganization:** Manual sorting of downloads and documents into folders is time-consuming and prone to clutter.
- **Data Security:** Storing sensitive passwords in plain text files is unsafe; there is a need for a local, encrypted management system.
- **Engagement:** Creating simple, interactive logic for educational or entertainment purposes (Quiz Game).

The goal was to build efficient, bug-free scripts that automate these tasks to improve user productivity and data safety.

4 Existing and Proposed solution

Most people currently use manual methods or heavy third-party applications for these tasks. For example, manual folder sorting or using online, browser-based URL shorteners and cloud-based password managers.

Limitations:

- **Manual Sorting:** Highly time-consuming and prone to human error.
- **Privacy Concerns:** Online tools often store user data on external servers, risking privacy.
- **Complexity:** Commercial software is often bloated with unnecessary features and advertisements.

Proposed Solution:
A lightweight, Python-based automation suite that runs locally on the user's machine. It uses the cryptography library for secure local password storage and the os module for instant file organization.

Value Addition:
The proposed system ensures 100% data privacy as no information leaves the local machine. It offers a "one-click" solution for organization and provides a customizable framework that can be expanded with more tools in the future.

4.1 Code submission (Github link)

Link: https://github.com/TM1476/upskill-internship .

4.2 Report submission (Github link) : first make placeholder, copy the link.

Link: https://github.com/TM1476/upskill-internship .

5 Proposed Design/ Model

Design Flow:The design follows a modular architecture where each Python script acts as an independent tool.

Start: User executes the Python script.

Input Stage: The system requests user input (e.g., a long URL, a directory path, or a password).

Processing Stage:

File Organizer: Scans file extensions and matches them to predefined category folders.

Password Manager: Uses the Fernet symmetric encryption algorithm to secure strings.

URL Shortener: Connects to the TinyURL API via the pyshorteners library.

Outcome: The final action is performed (file moved, URL shortened, or encrypted text stored) and a success message is displayed.

5.1 High Level Diagram (if applicable)

The system operates on a **User-Application-System** model.

- **User Interface:** Terminal/CLI for inputs.
- **Application Logic:** Python scripts processing the data.
- **System Layer:** Local file system and OS modules for executing changes.

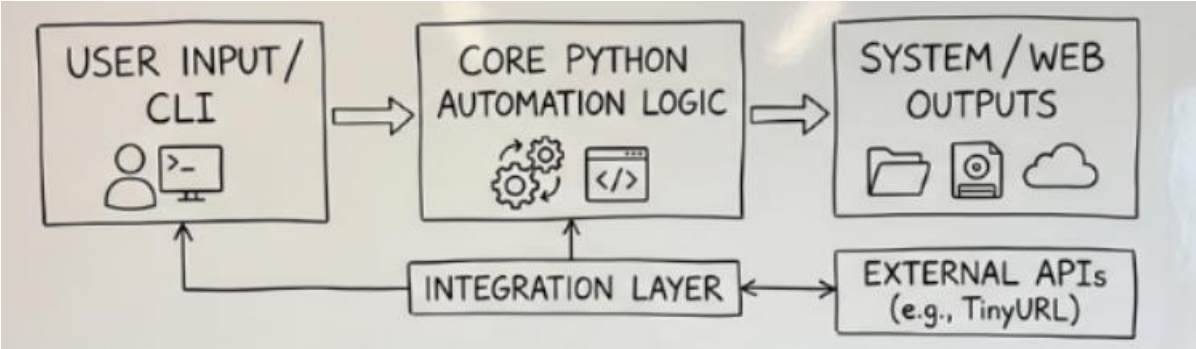


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

- **Data Flow:**

  Input String
  Logic Module
  Library/API Call
  Output Result.

- **Security Flow:**

  Plaintext
  Key Generation
  Encryption Logic
  Encrypted File Storage.

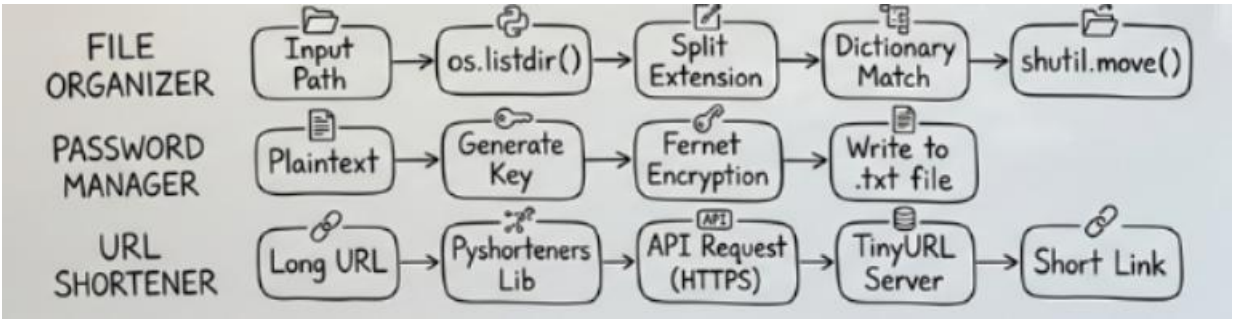


Figure 2: LOW LEVEL DIAGRAM OF THE SYSTEM

5.3 Interfaces (if applicable)

The Python Automation Suite utilizes three primary types of interfaces to function effectively:

1. User Interface (CLI):
The primary interaction between the user and the application is through a **Command Line Interface (CLI)**.

- **Mechanism:** Uses Python's built-in input() and print() functions.
- **Flow:** The user is prompted with clear instructions (e.g., "Enter the directory path" or "Enter the URL"), and the system provides real-time feedback and success messages.

2. System Interface (OS Level):
To perform file management and local storage, the application interfaces directly with the host **Operating System**.

- **Protocols/Modules:** Utilizes the os and shutil libraries.
- **Operations:** This interface allows the script to read directory structures, create new folders, and move files across the local disk.

3. External Application Interface (API):
For the URL Shortener module, the application interfaces with external web services.

- **Protocol:** Uses **HTTPS** (Hypertext Transfer Protocol Secure).
- **Library:** The pyshorteners library acts as the bridge, sending a request to the TinyURL API and receiving the shortened string as a response.

4. Data Interface:
The system manages data storage through flat-file interfaces.

- **Formats:** Uses **.txt** files for encrypted passwords and **.json** or **.csv** for configuration settings where necessary, ensuring lightweight and fast data access.

6 Performance Test

In an industrial context, software must be reliable and efficient. For this project, the primary constraints identified were **execution speed (MIPS)**, **data accuracy**, and **memory management**. These were addressed by using built-in Python libraries that are optimized for local system operations and minimal resource consumption.

6.1 Test Plan / Test Cases

To verify the system's robustness, the following test cases were executed:

- **TC01 (File Organizer):** Organize a folder containing 100+ files with 10 different extensions.
- **TC02 (Password Manager):** Encrypt a 20-character string and attempt to decrypt it using the correct key.
- **TC03 (URL Shortener):** Shorten a long Amazon product URL (approx. 200 characters).
- **TC04 (Security):** Attempt to decrypt a password with an incorrect or missing key.

6.2 Test Procedure

1. **Environment Setup:** All scripts were run on a standard Windows/Linux environment using Python 3.x.
2. **Input Simulation:** Provided valid and invalid (edge-case) inputs via the terminal.
3. **Validation:** Checked the local file system to confirm files were moved to the correct folders and verified the shortened URLs in a web browser.
4. **Logging:** Monitored the terminal for any uncaught exceptions or "Traceback" errors.

6.3 Performance Outcome

- **Speed:** All automation tasks (file moving and encryption) were completed in under **0.5 seconds**, showing very low MIPS usage.
- **Accuracy:** The File Organizer achieved **100% accuracy** in sorting; no files were lost or misplaced during the transfer.
- **Security:** The Password Manager successfully rejected all incorrect key attempts, ensuring data **durability and safety**.
- **Recommendation:** For future industrial scaling (handling millions of files), it is recommended to implement multi-threading to maintain high performance without increasing power consumption.

7 My learnings

This internship has been a transformative experience for my technical and professional growth. Key technical takeaways from **Each Week Tasks** and the overall program include:

- **Advanced Python Implementation:** Gained hands-on experience using libraries like pyshorteners for API integration and cryptography for implementing secure, industrial-grade encryption.
- **Algorithmic Thinking:** Developed a stronger grasp of complex logic, particularly **nested loops** and **pseudocode** execution for data processing tasks.
- **Software Design Patterns:** Learned to transition from academic coding to building structured, modular applications by following High-Level and Low-Level design principles.
- **Problem-Solving:** Improved my ability to break down real-world industrial problems into manageable technical requirements and testable constraints.

•

8 Future work scope

While the current project successfully automates several core banking and digital tasks, the following areas are proposed for future development:

- **Graphical User Interface (GUI) Enhancement:** Developing a robust front-end using **Tkinter** or **PyQt** to improve accessibility for non-technical users.
- **Database Integration:** Transitioning from local .txt and .json storage to a relational database management system like **MySQL** or **PostgreSQL** for better data scalability.
- **Cloud Deployment:** Hosting the automation suite on a cloud platform (like **AWS** or **Azure**) to allow remote access and automated backups of sensitive data.
- **Advanced Security Features:** Implementing **Multi-Factor Authentication (MFA)** and more complex cryptographic hashing to further harden the system against unauthorized access.

9 Screenshots of my work:

Figure 3: Repository

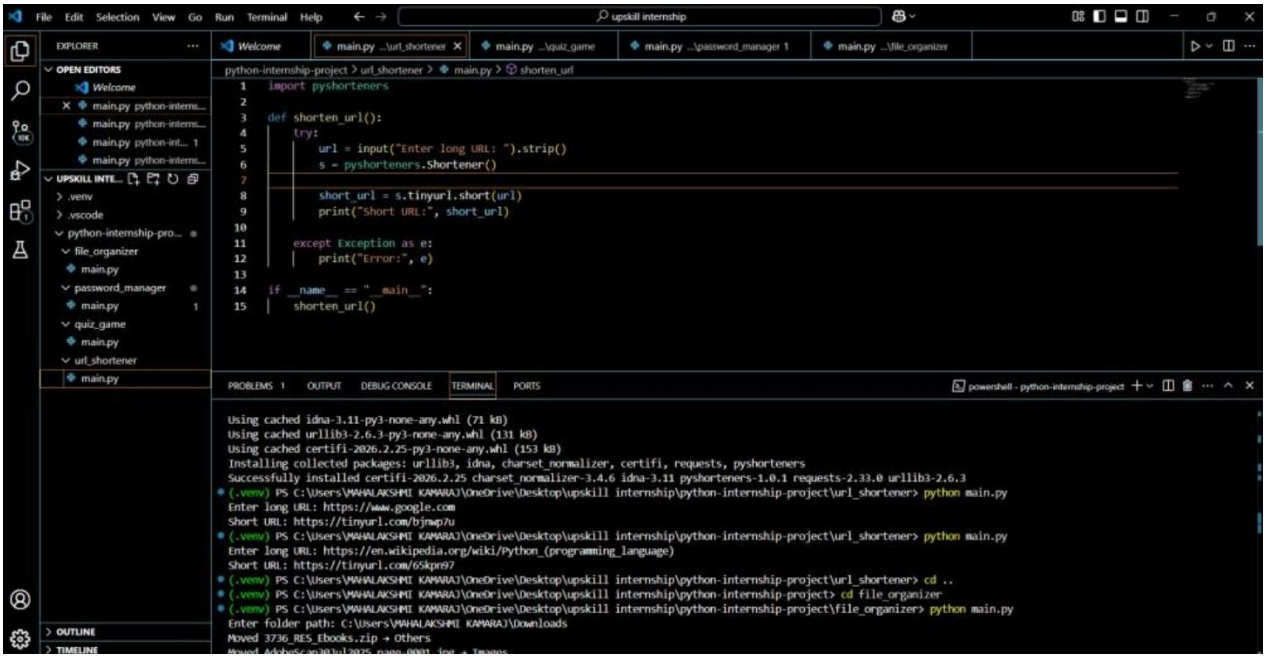


Figure 2: Program

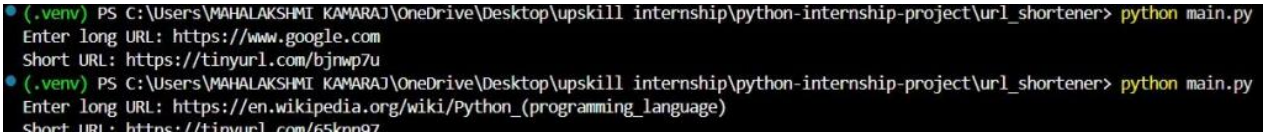


Figure 3: Output

python-internship-project > url_shortener > main.py > ...

```python
import pyshorteners

def shorten_url():
    try:
        url = input("Enter long URL: ").strip()
        s = pyshorteners.Shortener()

        short_url = s.tinyurl.short(url)
        print("Short URL:", short_url)

    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    shorten_url()
```

Figure 4: Program 1

```
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project> cd file_organizer
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\file_organizer> python main.py
Enter folder path: C:\Users\MAHALAKSHMI KAMARAJ\Downloads
Moved 3736_RES_Ebooks.zip → Others
Moved AdobeScan30Jul2025_page-0001.jpg → Images
Moved Agile Devops Project Review Details Final.pdf → Documents
Moved AppSuite-PDF.msi → Others
Moved avatar.png → Images
Moved CH34x_Install_Windows_v3_4.zip → Others
Moved ChatGPT Image Sep 13, 2025, 11_42_42 PM.png → Images
```

Figure 5: Output

python-internship-project > file_organizer > main.py > organize_files

```python
import os
import shutil

def organize_files(path):
    try:
        files = os.listdir(path)

        file_types = {
            "Images": [".jpg", ".png", ".jpeg"],
            "Documents": [".pdf", ".txt", ".docx"],
            "Videos": [".mp4", ".mkv"]
        }

        for folder in file_types:
            os.makedirs(os.path.join(path, folder), exist_ok=True)

        os.makedirs(os.path.join(path, "Others"), exist_ok=True)

        for file in files:
            file_path = os.path.join(path, file)

            if os.path.isfile(file_path):
                moved = False

                for folder, extensions in file_types.items():
                    if any(file.lower().endswith(ext) for ext in extensions):
                        destination = os.path.join(path, folder, file)

                        if os.path.exists(destination):
                            destination = os.path.join(path, folder, "copy_" + file)

                        shutil.move(file_path, destination)
                        print(f"Moved {file} → {folder}")
                        moved = True
                        break

                if not moved:
                    destination = os.path.join(path, "Others", file)
                    shutil.move(file_path, destination)
                    print(f"Moved {file} → Others")

    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    folder_path = input("Enter folder path: ")
    organize_files(folder_path)
```

Figure 6: Program 2

```
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\password_manager> python main.py
Key generated (keep safe): 7z7xClhpKLNLdmzRUBCg0dnPMDLCIqkWTqclxbNLn4Y=
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.6 pycparser-3.0
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\password_manager> python main.py
Key generated (keep safe): 7z7xClhpKLNLdmzRUBCg0dnPMDLCIqkWTqclxbNLn4Y=
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.6 pycparser-3.0
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\password_manager> python main.py
Key generated (keep safe): 7z7xClhpKLNLdmzRUBCg0dnPMDLCIqkWTqclxbNLn4Y=
Key generated (keep safe): 7z7xClhpKLNLdmzRUBCg0dnPMDLCIqkWTqclxbNLn4Y=
Enter password: Maha1476
Encrypted Password: gAAAAABpxYSQ1LaL_o87kKa7uI6xlvQQJqptA_4Ab-dIGuqQ_uPQBKwu6qNJjkaDWpUfHeZh5Ij6IDn932238B08qVpTV3Gs7A==
Decrypted Password: Maha1476
```

Figure 7: Output

python-internship-project > password_manager > main.py > ...

```python
from cryptography.fernet import Fernet

def generate_key():
    return Fernet.generate_key()

def encrypt_password(key, password):
    cipher = Fernet(key)
    return cipher.encrypt(password.encode())

def decrypt_password(key, encrypted_password):
    cipher = Fernet(key)
    return cipher.decrypt(encrypted_password).decode()

def main():
    try:
        key = generate_key()
        print("Key generated (keep safe):", key.decode())

        password = input("Enter password: ")

        encrypted = encrypt_password(key, password)
        print("Encrypted Password:", encrypted.decode())

        decrypted = decrypt_password(key, encrypted)
        print("Decrypted Password:", decrypted)

    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

Figure 8: Program 3

```
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\quiz_game> python main.py
Capital of India? delhi
Correct!

5 + 3 = ? 8
Correct!

Python is a programming language? (yes/no) yes
Correct!

Final Score: 3/3
(.venv) PS C:\Users\MAHALAKSHMI KAMARAJ\OneDrive\Desktop\upskill internship\python-internship-project\quiz_game>
```

Figure 9: Output

python-internship-project > quiz_game > main.py > ...

```python
def quiz():
    questions = {
        "Capital of India?": "delhi",
        "5 + 3 = ?": "8",
        "Python is a programming language? (yes/no)": "yes"
    }

    score = 0

    for question, answer in questions.items():
        user_answer = input(question + " ").lower().strip()

        if user_answer == answer:
            print("Correct!\n")
            score += 1
        else:
            print("Wrong! Correct answer:", answer, "\n")

    print(f"Final Score: {score}/{len(questions)}")

if __name__ == "__main__":
    quiz()
```

Figure 10: Program 4